```
 GNU nano 8.7.1                                          wait_example.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid1, pid2, pid3;
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n\n", getpid());

    pid1 = fork();

    if (pid1 == 0)
    {
        printf("Child 1 started. PID = %d\n", getpid());
        sleep(3);
        printf("Child 1 completed\n");
        exit(1);
    }

    pid2 = fork();

    if (pid2 == 0)
    {
        printf("Child 2 started. PID = %d\n", getpid());
        sleep(2);
        printf("Child 2 completed\n");
        exit(2);
    }

    pid3 = fork();

    if (pid3 == 0)
```

```
    {
        printf("Child 2 started. PID = %d\n", getpid());
        sleep(2);
        printf("Child 2 completed\n");
        exit(2);
    }

    pid3 = fork();

    if (pid3 == 0)
    {
        printf("Child 3 started. PID = %d\n", getpid());
        sleep(1);
        printf("Child 3 completed\n");
        exit(3);
    }

    printf("\nParent waiting for child processes...\n");

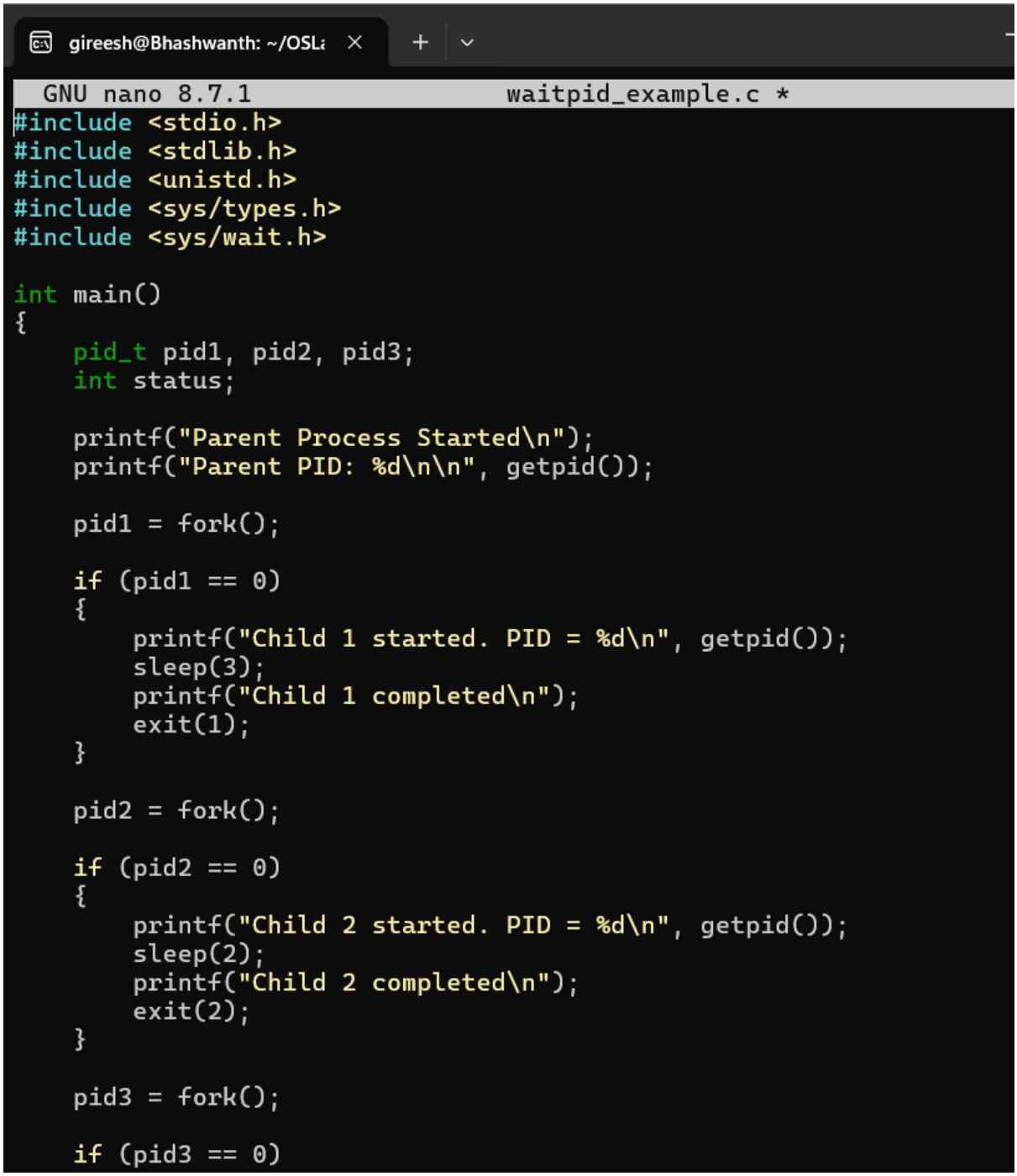
    for (int i = 0; i < 3; i++)
    {
        pid_t completed_pid = wait(&status);

        printf("Parent collected Child PID %d\n", completed_pid);

        if (WIFEXITED(status))
        {
            printf("Exit status = %d\n", WEXITSTATUS(status));
        }
    }

    printf("\nAll child processes completed.\n");

    return 0;
}
```


gireesh@Bhashwanth: ~/OSL
GNU nano 8.7.1 waitpid_example.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main()
{
pid_t pid1, pid2, pid3;
int status;
printf("Parent Process Started\n");
printf("Parent PID: %d\n\n", getpid());
pid1 = fork();
if (pid1 == 0)
{
printf("Child 1 started. PID = %d\n", getpid());
sleep(3);
printf("Child 1 completed\n");
exit(1);
}
pid2 = fork();
if (pid2 == 0)
{
printf("Child 2 started. PID = %d\n", getpid());
sleep(2);
printf("Child 2 completed\n");
exit(2);
}
pid3 = fork();
if (pid3 == 0)

```c
    if (pid3 == 0)
    {
        printf("Child 3 started. PID = %d\n", getpid());
        sleep(1);
        printf("Child 3 completed\n");
        exit(3);
    }

    printf("\nParent waiting specifically for Child 1...\n");

    waitpid(pid1, &status, 0);

    printf("Child 1 collected. PID = %d\n", pid1);

    if (WIFEXITED(status))
    {
        printf("Child 1 exit status = %d\n", WEXITSTATUS(status));
    }

    printf("\nWaiting for Child 2...\n");

    waitpid(pid2, &status, 0);

    printf("Child 2 collected. PID = %d\n", pid2);

    printf("\nWaiting for Child 3...\n");

    waitpid(pid3, &status, 0);

    printf("Child 3 collected. PID = %d\n", pid3);

    printf("\nAll child processes completed.\n");

    return 0;
}
```

gireesh@Bhashwanth: ~/OSLa

GNU nano 8.7.1 zombie.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started\n");
        printf("Child PID = %d\n", getpid());

        sleep(2);

        printf("Child process terminating\n");

        exit(0);
    }
    else
    {
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent will sleep for 30 seconds.\n");
        printf("During this time, the child may become a zombie.\n");

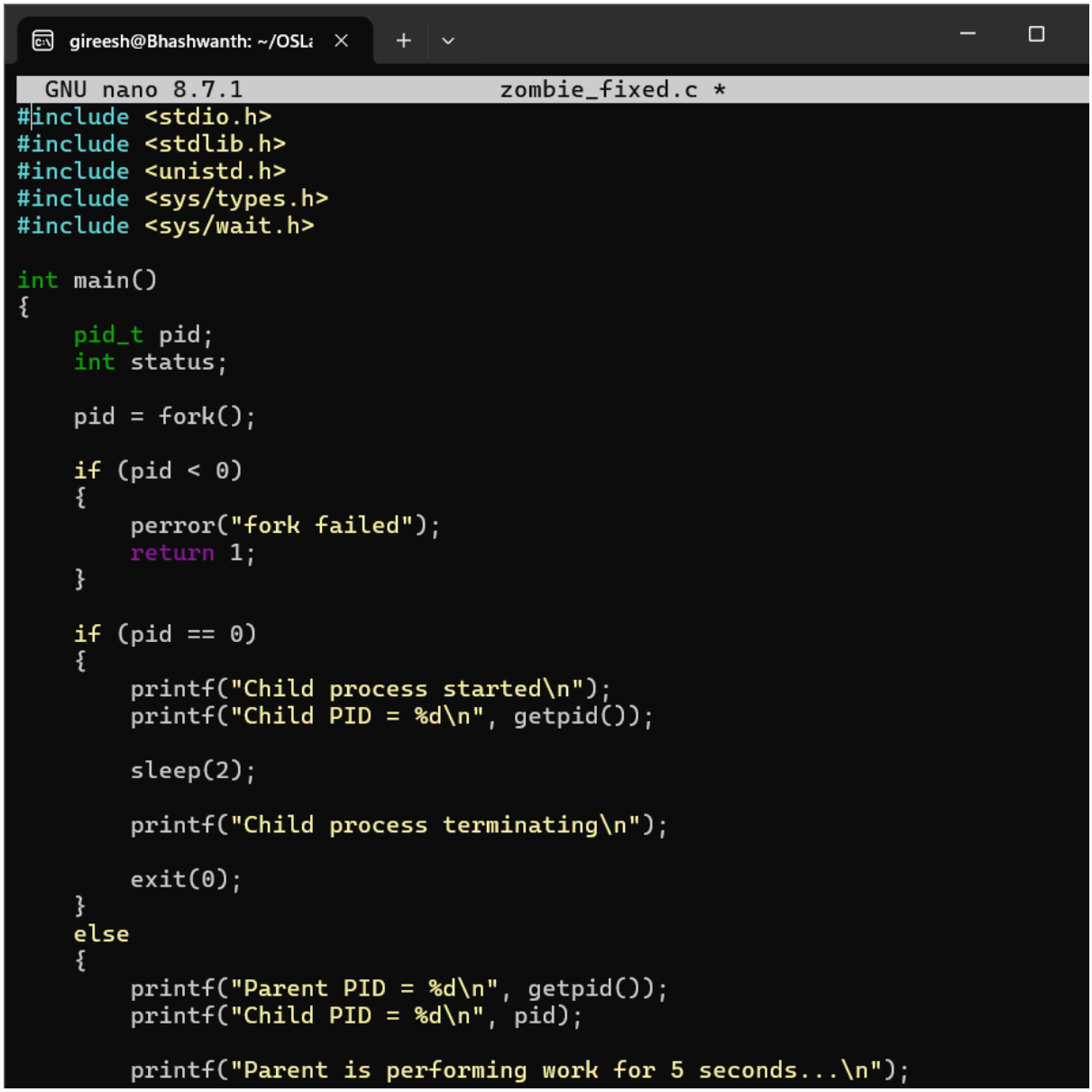
        sleep(30);
```

```c
        printf("During this time, the child may become a zombie.\n

        sleep(30);

        printf("Parent terminating without wait().\n");
    }

    return 0;
}
```


gireesh@Bhashwanth: ~/OSLa
GNU nano 8.7.1 zombie_fixed.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main()
{
pid_t pid;
int status;
pid = fork();
if (pid < 0)
{
perror("fork failed");
return 1;
}
if (pid == 0)
{
printf("Child process started\n");
printf("Child PID = %d\n", getpid());
sleep(2);
printf("Child process terminating\n");
exit(0);
}
else
{
printf("Parent PID = %d\n", getpid());
printf("Child PID = %d\n", pid);
printf("Parent is performing work for 5 seconds...\n");

```
    else
    {
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent is performing work for 5 seconds...\n");

        sleep(5);

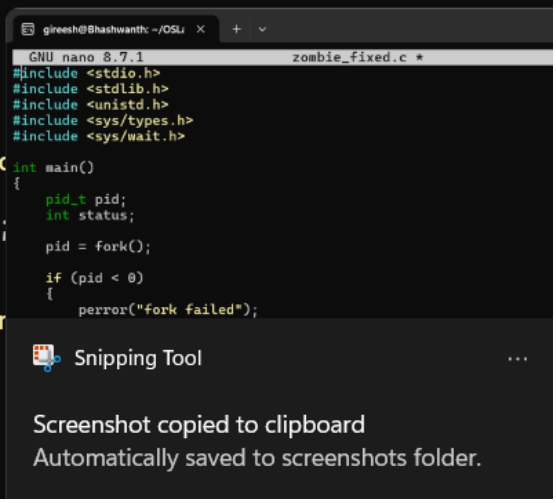
        printf("Parent calling waitpid() to collect child...\n");

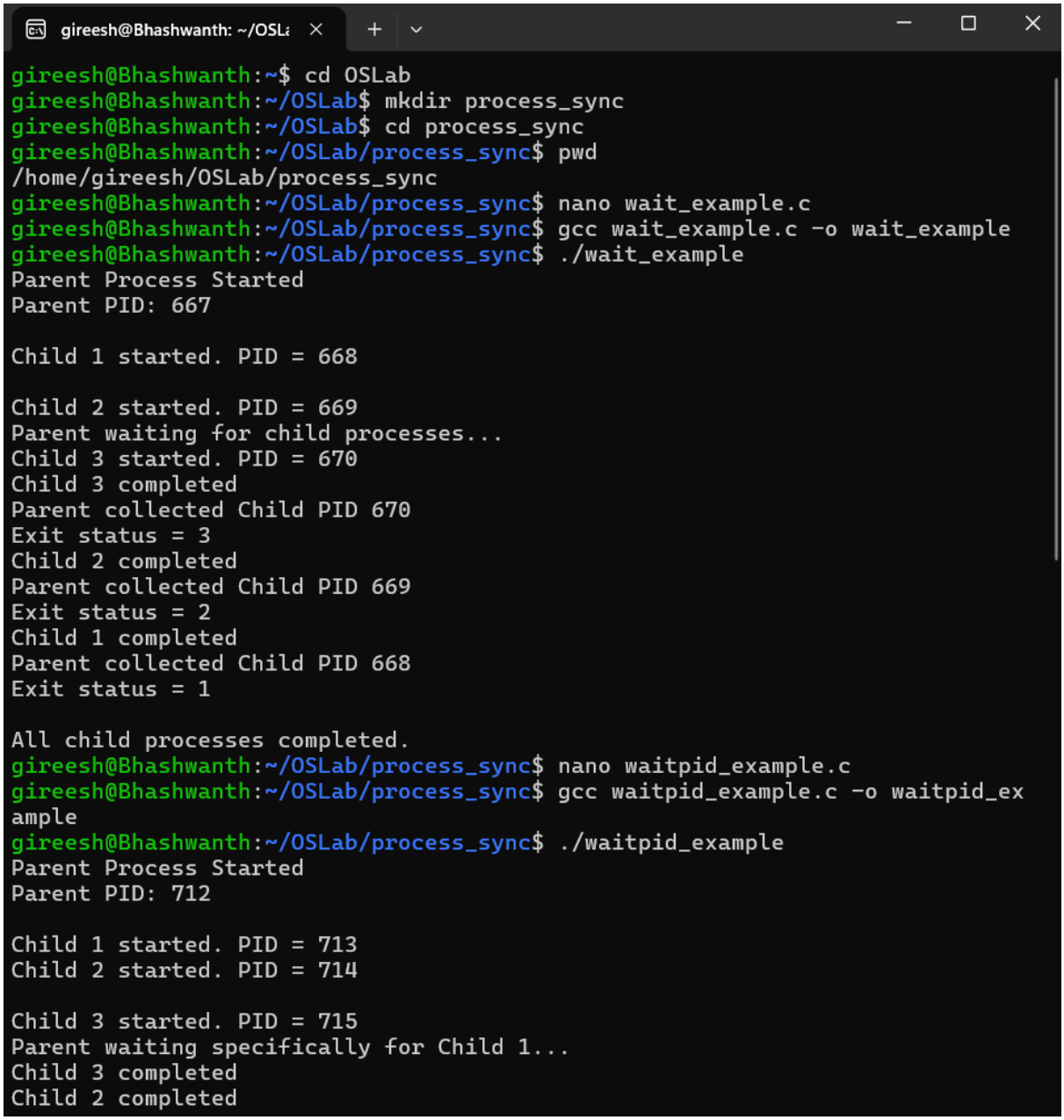
        waitpid(pid, &status, 0);

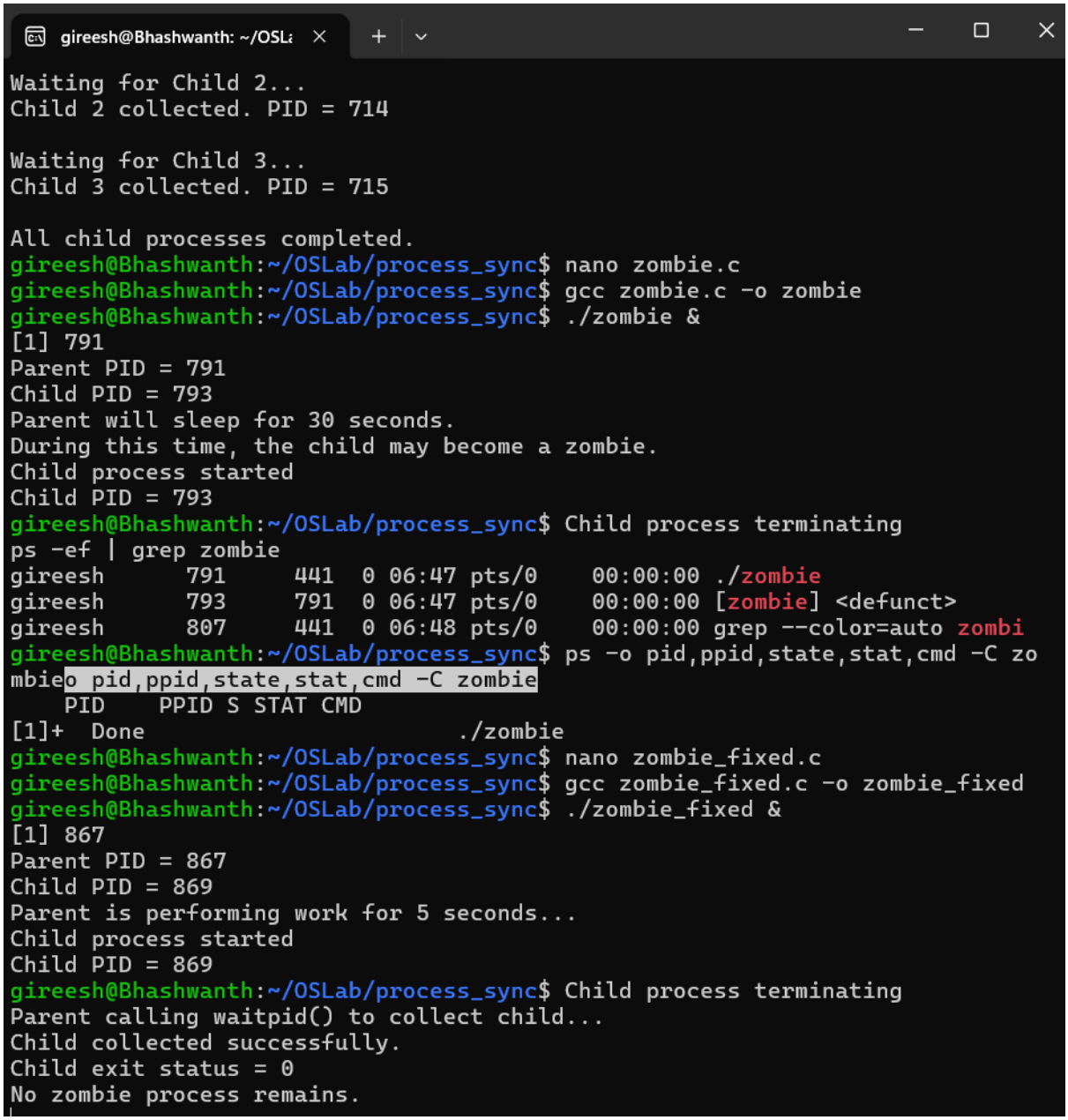
        if (WIFEXITED(status))
        {
            printf("Child collected succ
            printf("Child exit status =
                   WEXITSTATUS(status));
        }

        printf("No zombie process remair
    }

    return 0;
}
```




gireesh@Bhashwanth: ~/OSLa
gireesh@Bhashwanth:~$ cd OSLab
gireesh@Bhashwanth:~/OSLab$ mkdir process_sync
gireesh@Bhashwanth:~/OSLab$ cd process_sync
gireesh@Bhashwanth:~/OSLab/process_sync$ pwd
/home/gireesh/OSLab/process_sync
gireesh@Bhashwanth:~/OSLab/process_sync$ nano wait_example.c
gireesh@Bhashwanth:~/OSLab/process_sync$ gcc wait_example.c -o wait_example
gireesh@Bhashwanth:~/OSLab/process_sync$ ./wait_example
Parent Process Started
Parent PID: 667
Child 1 started. PID = 668
Child 2 started. PID = 669
Parent waiting for child processes...
Child 3 started. PID = 670
Child 3 completed
Parent collected Child PID 670
Exit status = 3
Child 2 completed
Parent collected Child PID 669
Exit status = 2
Child 1 completed
Parent collected Child PID 668
Exit status = 1
All child processes completed.
gireesh@Bhashwanth:~/OSLab/process_sync$ nano waitpid_example.c
gireesh@Bhashwanth:~/OSLab/process_sync$ gcc waitpid_example.c -o waitpid_ex
ample
gireesh@Bhashwanth:~/OSLab/process_sync$ ./waitpid_example
Parent Process Started
Parent PID: 712
Child 1 started. PID = 713
Child 2 started. PID = 714
Child 3 started. PID = 715
Parent waiting specifically for Child 1...
Child 3 completed
Child 2 completed


gireesh@Bhashwanth: ~/OSLa
Waiting for Child 2...
Child 2 collected. PID = 714
Waiting for Child 3...
Child 3 collected. PID = 715
All child processes completed.
gireesh@Bhashwanth:~/OSLab/process_sync$ nano zombie.c
gireesh@Bhashwanth:~/OSLab/process_sync$ gcc zombie.c -o zombie
gireesh@Bhashwanth:~/OSLab/process_sync$ ./zombie &
[1] 791
Parent PID = 791
Child PID = 793
Parent will sleep for 30 seconds.
During this time, the child may become a zombie.
Child process started
Child PID = 793
gireesh@Bhashwanth:~/OSLab/process_sync$ Child process terminating
ps -ef | grep zombie
gireesh 791 441 0 06:47 pts/0 00:00:00 ./zombie
gireesh 793 791 0 06:47 pts/0 00:00:00 [zombie] <defunct>
gireesh 807 441 0 06:48 pts/0 00:00:00 grep --color=auto zombi
gireesh@Bhashwanth:~/OSLab/process_sync$ ps -o pid,ppid,state,stat,cmd -C zo
mbieo pid,ppid,state,stat,cmd -C zombie
PID PPID S STAT CMD
[1]+ Done ./zombie
gireesh@Bhashwanth:~/OSLab/process_sync$ nano zombie_fixed.c
gireesh@Bhashwanth:~/OSLab/process_sync$ gcc zombie_fixed.c -o zombie_fixed
gireesh@Bhashwanth:~/OSLab/process_sync$ ./zombie_fixed &
[1] 867
Parent PID = 867
Child PID = 869
Parent is performing work for 5 seconds...
Child process started
Child PID = 869
gireesh@Bhashwanth:~/OSLab/process_sync$ Child process terminating
Parent calling waitpid() to collect child...
Child collected successfully.
Child exit status = 0
No zombie process remains.